B24-L6 lab: Train a CNN on handwritten digits and compare it with a dense network

Platform: Laptop

Work in `labs/cnn/` in this repository. It uses scikit-learn's bundled 8 by 8 handwritten digits, so nothing downloads. You write the CNN steps from module B24: the channel dimension, a shift augmentation, the model, a confusion matrix and per-class recall. The training script trains your CNN and a dense network on the same split, with early stopping. You need PyTorch 2.15 on the laptop. When the CNN beats the dense network, push the lab to your own GitHub repository: it is the roadmap's CNN evidence for months 4 to 6.

1. Run `cd labs/cnn`, create a virtual environment, then `pip install -e .` and `pip install pytest`. Run `pytest`: most tests fail with NotImplementedError. Read `README.md` and every file in `src/digits_cnn/`.
2. In `src/digits_cnn/vision.py`, write `add_channel` (lessons B24-L2 and B24-L4) and `random_shift` (lesson B24-L4). Run `pytest tests/test_vision.py -k "channel or shift"`.
3. Write `confusion` and `recall_by_class` in the same file (lesson B24-L6).
4. In `src/digits_cnn/models.py`, write `make_cnn` (lessons B24-L1 to B24-L3). Run `pytest`: every test passes.
5. Run `python -m digits_cnn train`. It prints validation accuracy for the dense network and the CNN, and the CNN's recall for each digit. Which digits does it confuse?
6. Run `python -m digits_cnn train --augment`, which shifts training images by up to one pixel. Compare both models with and without it, and explain the result (lesson B24-L4).
7. Write your README: the data, both models, their parameter counts, the comparison, the confusion matrix, and what

augmentation changed. Push the folder to a new GitHub repository.

Notes